Let's list *all* our containers, including stopped ones, by adding the -a option:

```
$ docker ps -a
CONTAINER ID  IMAGE     COMMAND  CREATED  STATUS   PORTS  NAMES
974e2bcb8266  ruby:2.6  "ruby …  1 seco…  Exited…         dazzling_ba…
7f8d7dddd6b5  ruby:2.6  "ruby …  3 seco…  Exited…         hungry_heis…
```

Here you can see two containers: one for each time we ran the Ruby command. However, we no longer have any need for them; we can delete them with:

```
$ docker rm <container id> [<container id2> ...]
```

Your container IDs will differ from mine as they are randomly generated. To delete these containers, I'd run:

```
$ docker rm 974e2bcb8266 7f8d7dddd6b5
```

However, you will need to substitute your container IDs to run this. Do this now to clean up your containers.

In the future, when creating a container we have no further use for, we can use the --rm option, which tells Docker to delete the container after it completes. The fancy word for short-lived containers that are deleted after they've served their purpose is *ephemeral*, but I prefer the word *throwaway*. Here's how we'd run our Ruby script in a throwaway container:

```
$ docker run --rm ruby:2.6 ruby -e "puts :hello"
```

This is a fairly common pattern, and you'll see it throughout the book.

Generating a New Rails App Without Ruby Installed

Running a Ruby script was cool, but what else can we do?

Wouldn't it be nice to start using Docker for some "real-world" tasks? Let's imagine we want to create a new Rails project (that's not so far-fetched…we are Ruby developers after all). Can we do that? You bet.

We're going to want to run multiple commands in succession in a container in order to generate the Rails project. We *could* craft a really long, ugly docker run that executes the instructions one after another. However, that's going to be hard to comprehend.

Instead, we can do something a little different. We can start a container running an interactive Bash shell. When we do this, we literally get a terminal session running *inside* the container. From there, we can run as many commands as we like, much like if we had a local Bash session. This is a very useful trick to have up your sleeve.